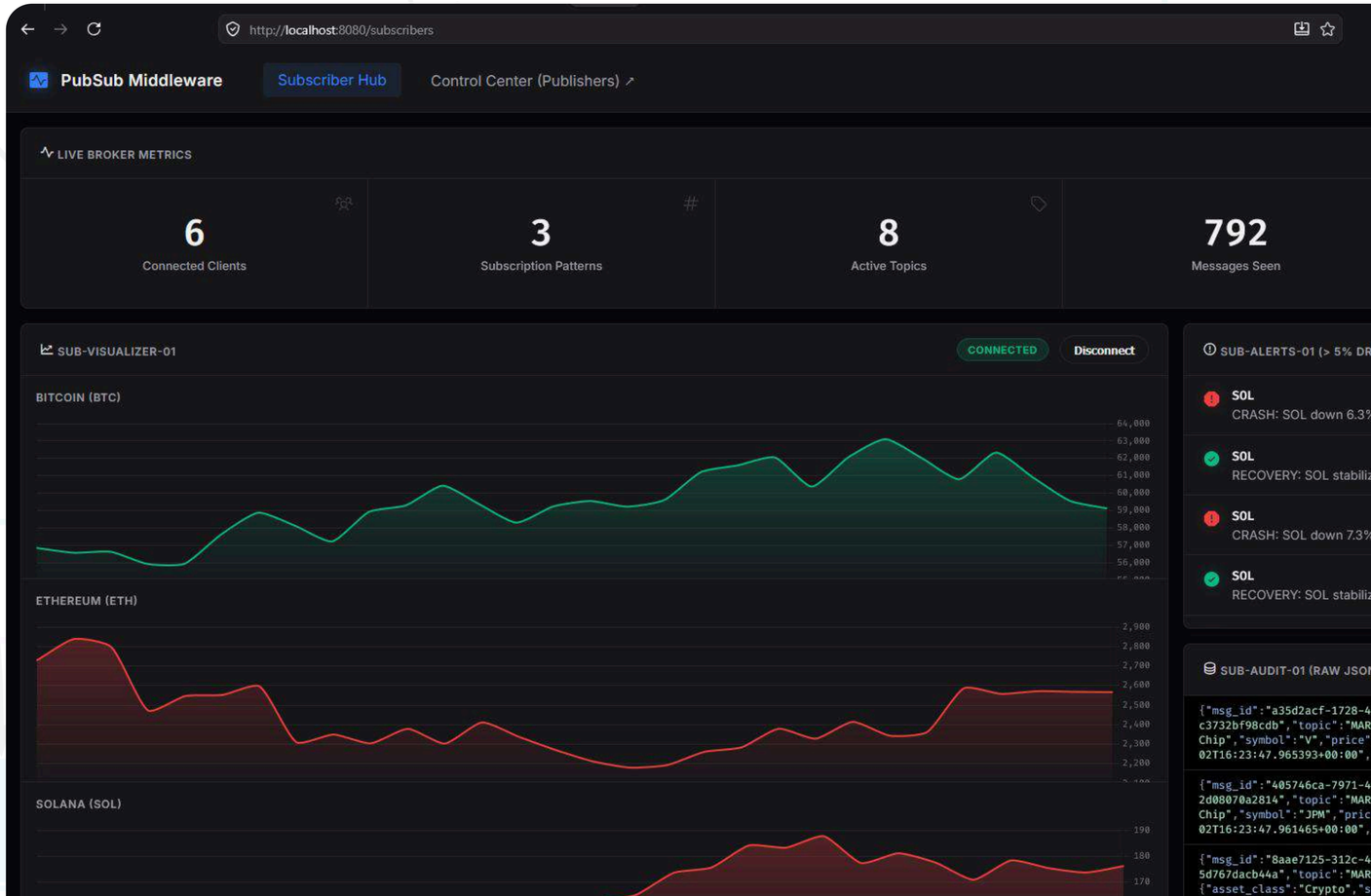
Distributed Publish-Subscribe Messaging Middleware

A Multi-Tiered Distributed Messaging System for Real-Time Market Data and Fault-Tolerant IPC



Aquino, Dallas A.
Buñag, Frederick Jibril L.
Carbonell, Ethan Jed V.
Corpes, Vincent L. Jr.

BS Computer Science 3A AI

Mr. Cyreneo S. Dofitas
CCS 231 - Parallel & Distributed Computing

May 2026

The Core Concept

Publisher

The Producer

Broker

The Middleman

Subscriber

The Consumer

SYSTEM OVERVIEW

5 PILLARS OF OUR SYSTEM

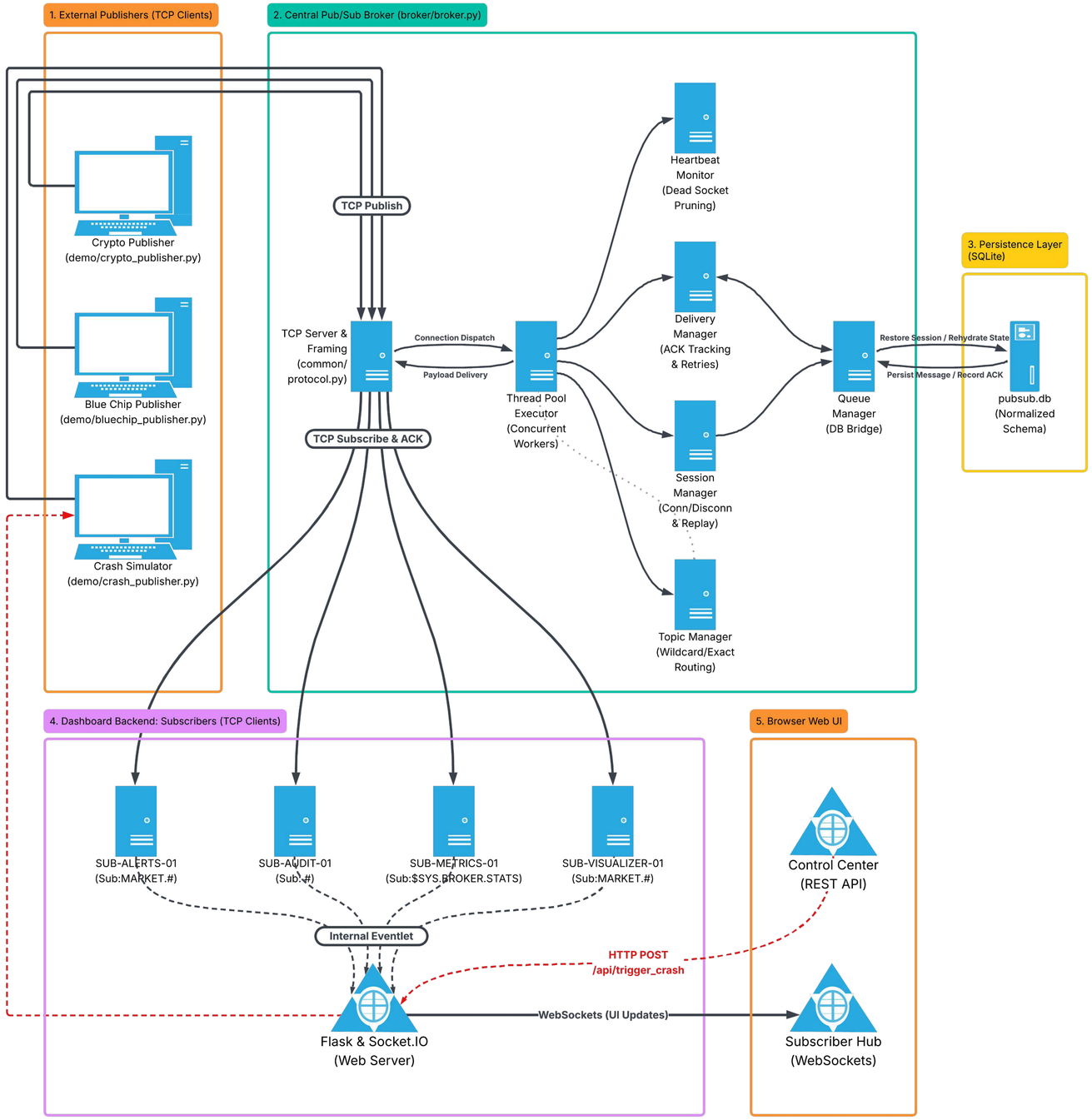
1 External Publishers
Independent Python processes acting as data producers.

2 Central Pub/Sub Broker
The core TCP server responsible for routing, state management, and connection handling.

3 Persistence Layer
The database ensuring durable message queuing and fault tolerance.

Dashboard Backend
Internal TCP clients that consume data and bridge it to the web server.

5 Browser Web UI
The real-time visual frontend for monitoring and system control.



PHASE 1 : Data Generation

THE PUBLISHERS

1 Crypto Publisher

Simulates volatile digital assets.

TOPICS (Cryptocurencies)

- BTC Bitcoin
- ETH Etherium
- SOL Solana
- DOGE Dogecoin

2 Blue Chip Publisher

Simulates stable stocks using mean-reversion.

TOPICS (Stocks)

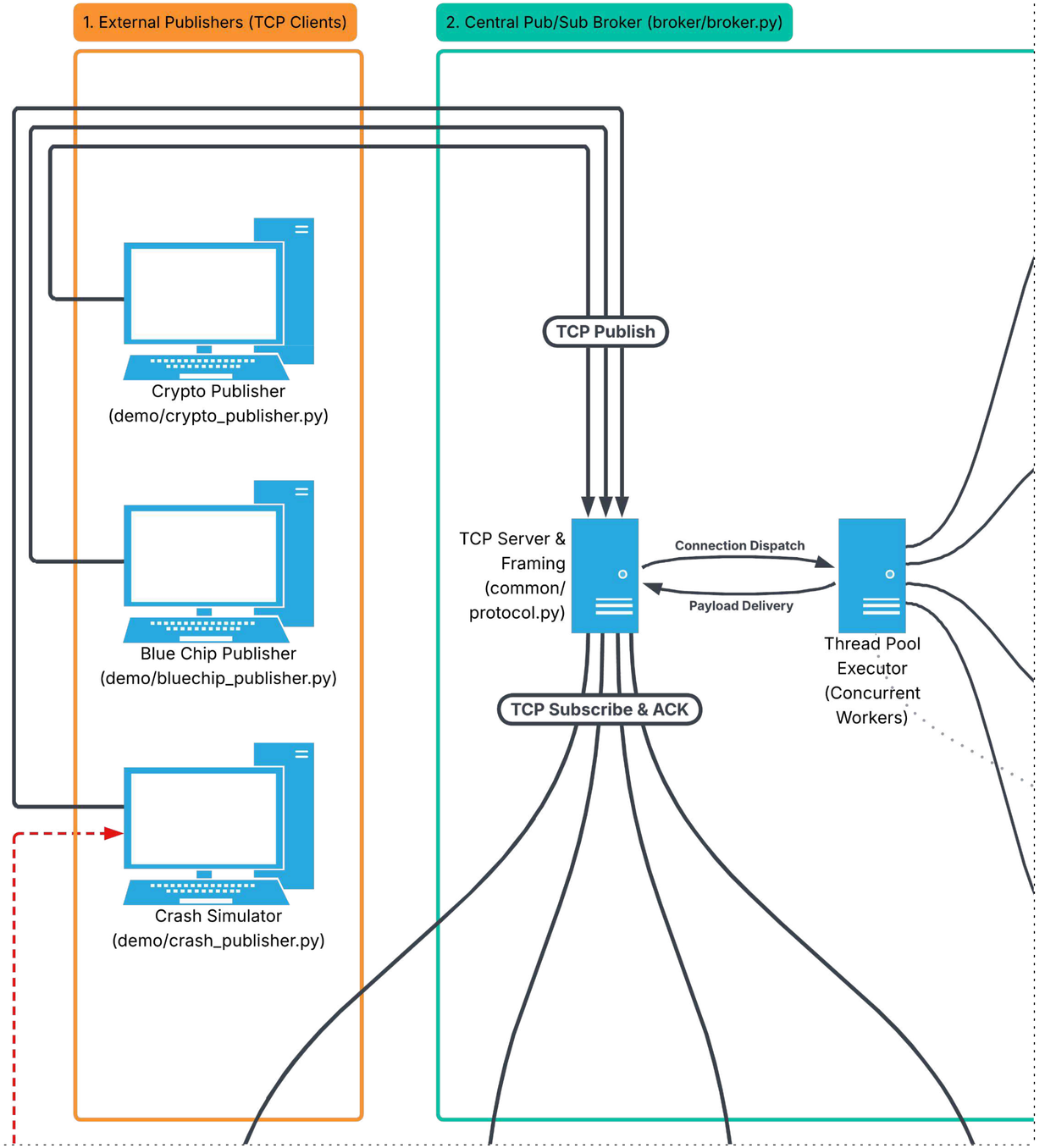
- AAPL Apple Inc.
- MSFT Microsoft Corp
- JPM JPMorgan Chase & Co
- V VISA Inc.

3 Crash Simulator

A one-shot emergency testing script.

TCP Framing

4-byte length prefix prevents TCP stream fragmentation.



PHASE 2: Ingestion & Routing

THE BROKER

TCP Server & Thread Pool

Avoids blocking via 50 concurrent background worker threads.

Topic Manager

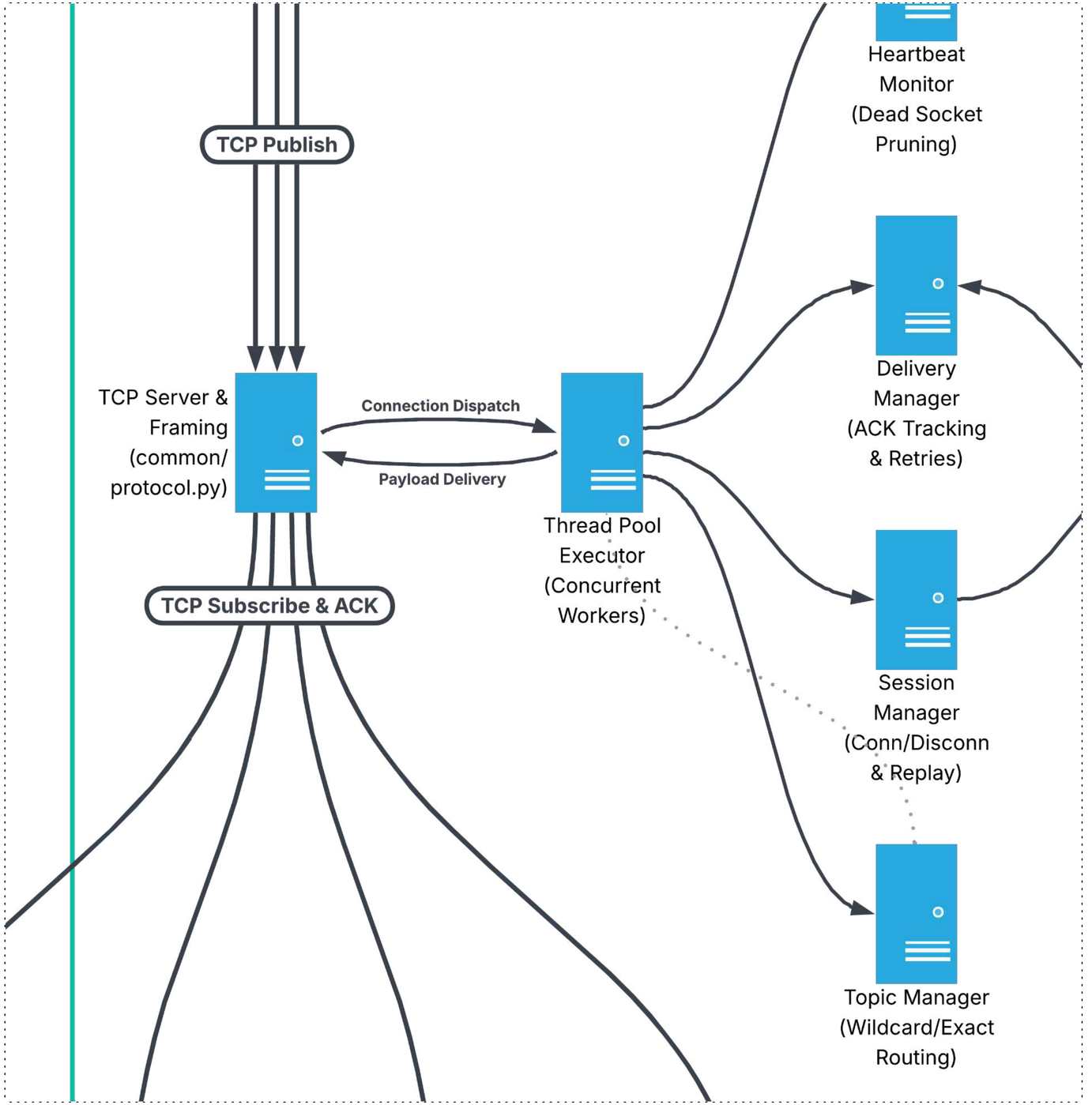
Evaluates exact string matches and Wildcards.

WILDCARDS

MARKET.#	Matches all branches of financial data
#	The global catch-all

EXACT STRING MATCHES

$SYS.BROKER.STATS	Matches only internal broker health metrics letter-for-letter



PHASE 3:
At-Least-Once Guarantees

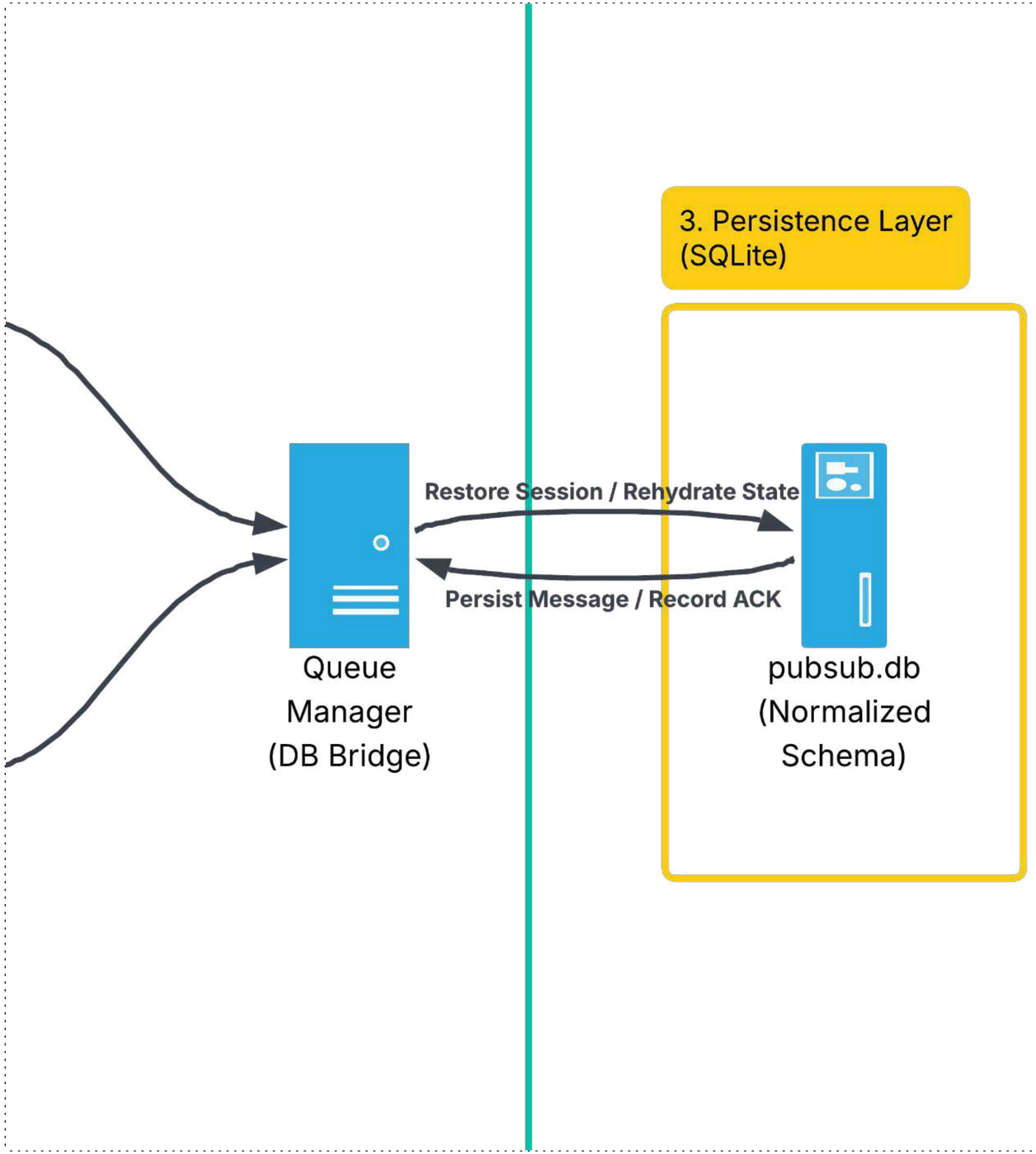
PERSISTENCE

Delivery Manager & Queue Manager
Intercepts messages before network transmission.

SQLite Database
persistence/db.py

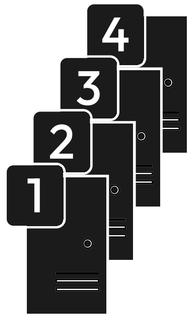
SCHEMA

messages table	Stores the heavy JSON payload exactly once.
delivery_status table	Stores lightweight rows for each subscriber marked as **PENDING.**



PHASE 4: Consumption & Web UI

THE DASHBOARD



Dashboard Subscribers

Python threads listening for specific topics

SUBSCRIBERS

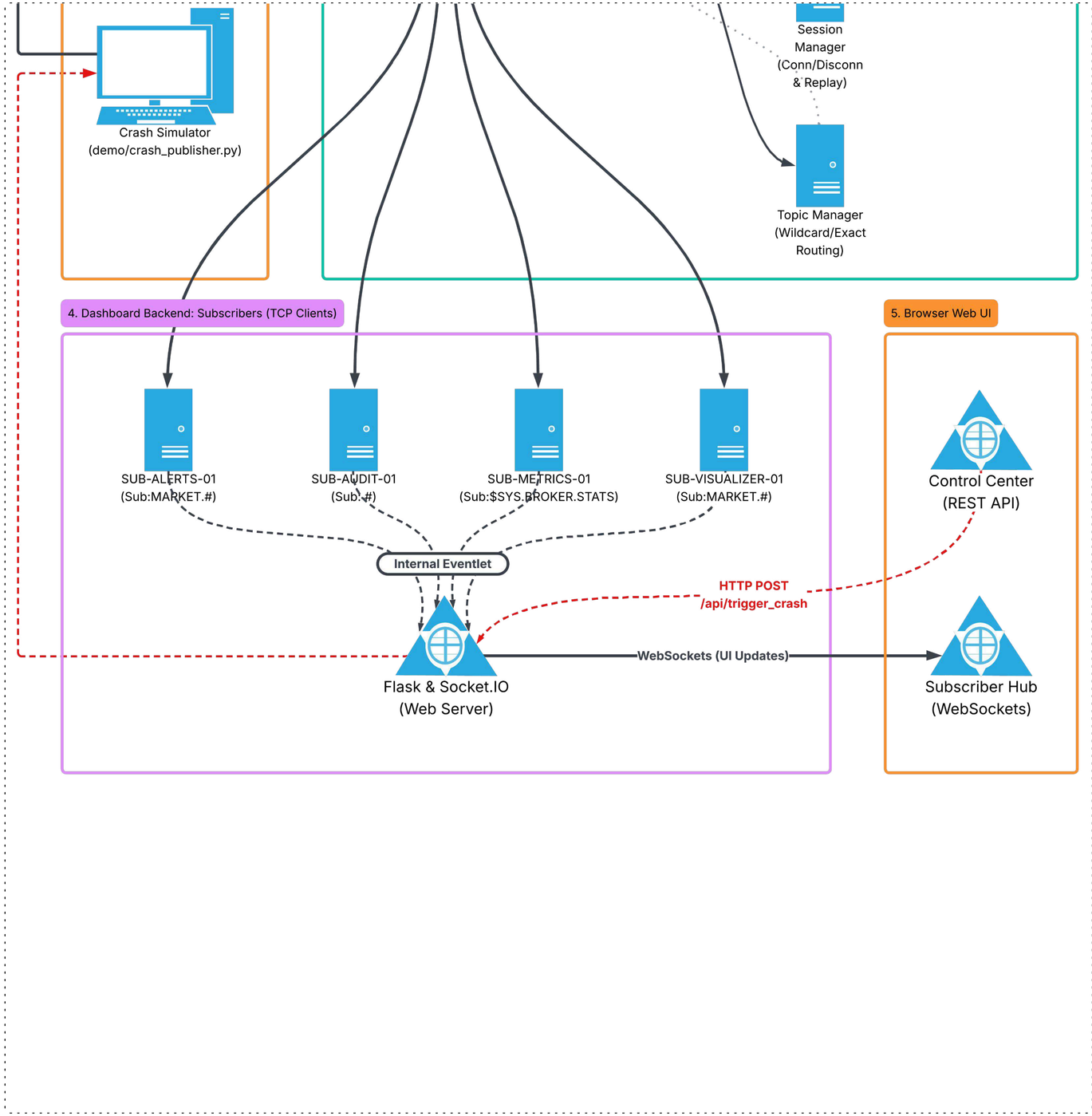
SUB-VISUALIZER-01	Feeds live line charts.
SUB-ALERTS-01	Edge-triggered >5% drop alerts.
SUB-AUDIT-01	Raw JSON compliance firehose.
SUB-METRICS-01	Live broker health stats.

Flask & Socket.IO

Broadcasts Python data to the browser via WebSockets.

The ACK Process

Subscriber receives data -> Sends ACK -> Broker deletes **PENDING** database row.



PHASE 5: Fault Tolerance

WHEN THINGS BREAK

Heartbeat Monitor

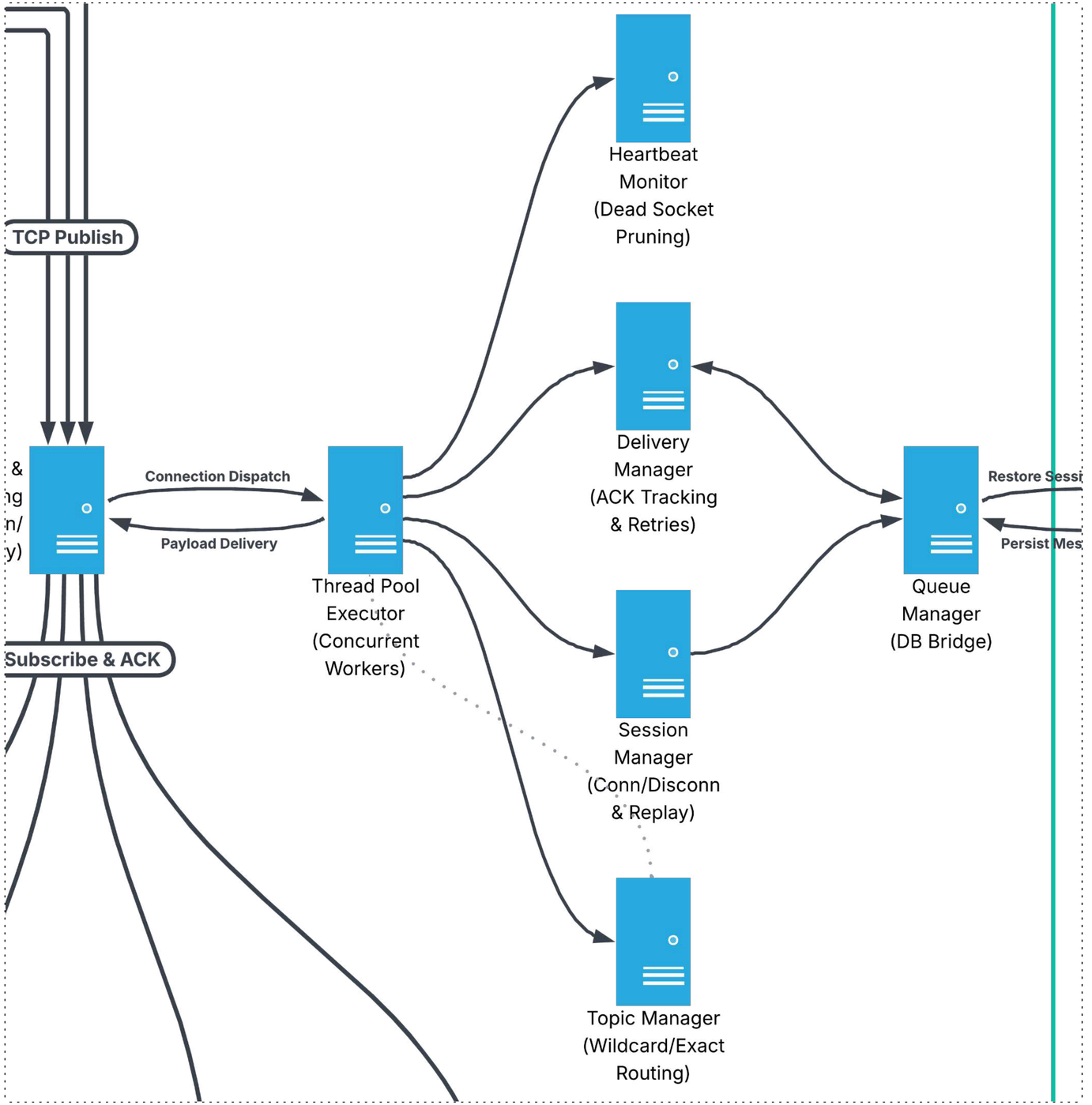
Clients send "pings" every 10 seconds.

Missing 2 pings (20s) = Broker forcefully severs the "zombie" TCP socket.

Session & Delivery Managers

Offline subscribers cause messages to safely queue as **PENDING**.

Upon reconnection, missed messages are instantly replayed.



PHASE 6:
The Control Loop

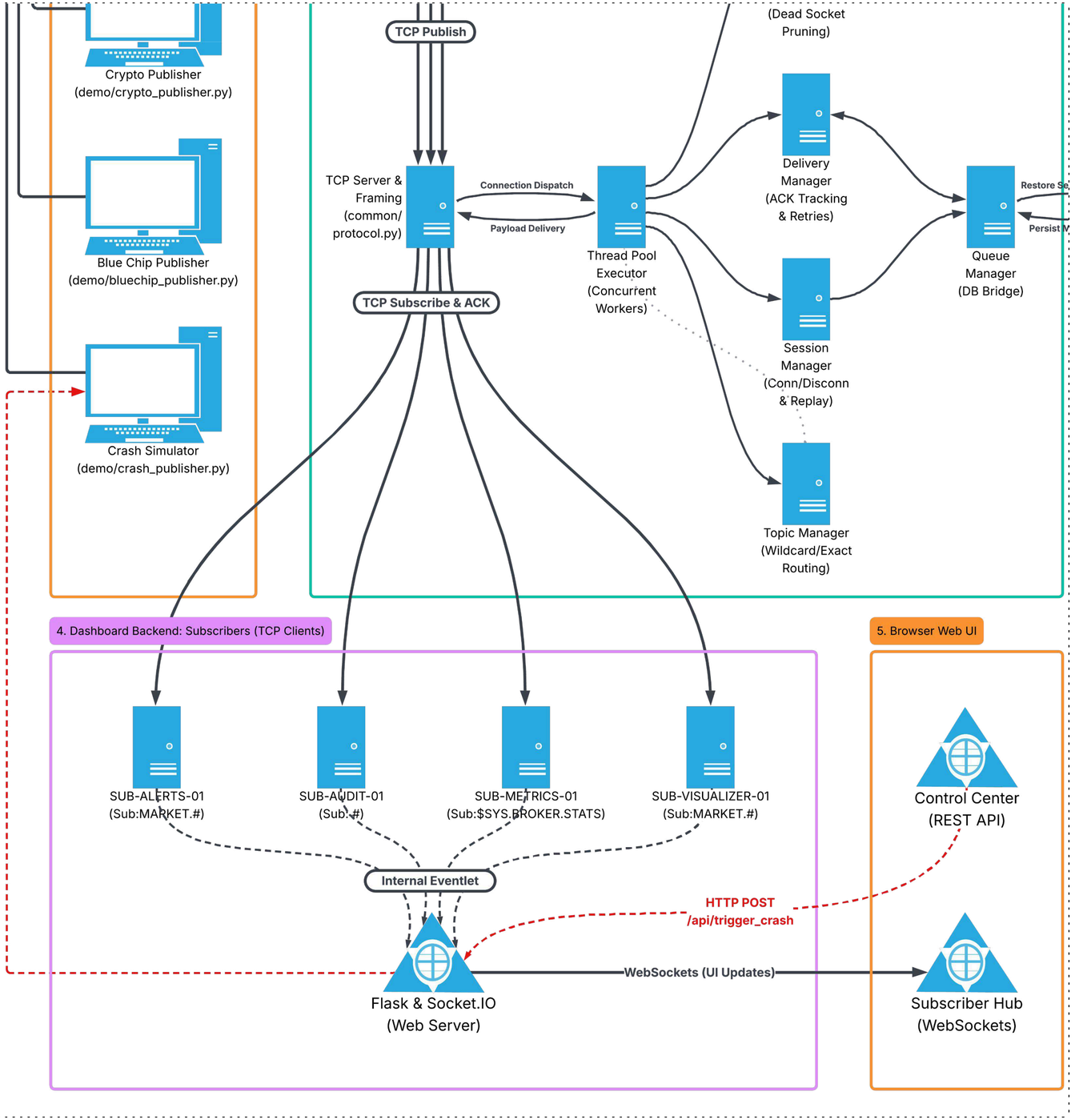
FLASH CRASH SIMULATOR

1 Trigger a topic to crash
Admin triggers an HTTP POST via the Web UI.

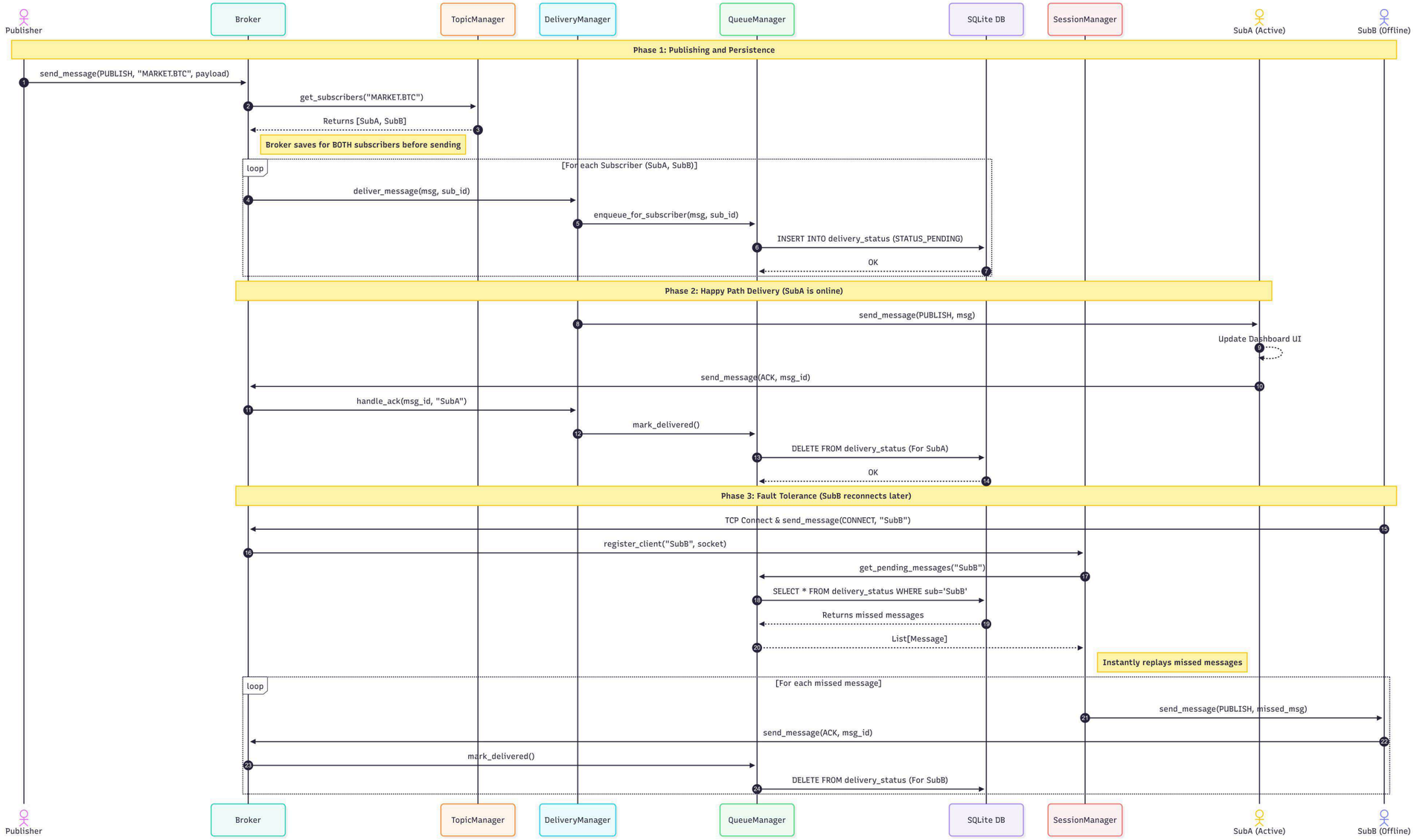
2 Spawn the crash
Flask tells the OS to spawn crash_publisher.py.

3 Execute the crash

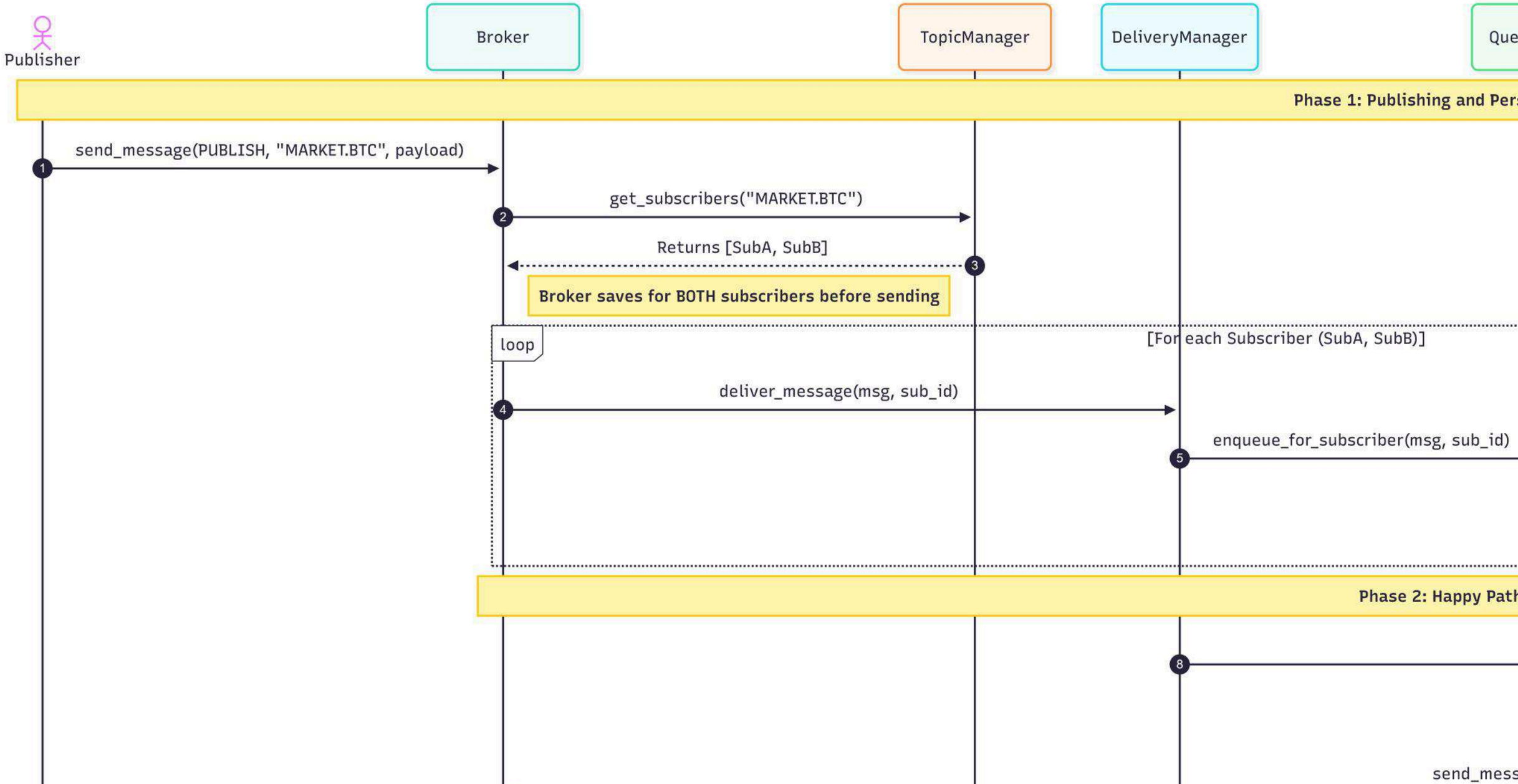
- Connects to Broker
- Fires negative price spike
- Disconnects
- Subscribers pick it up



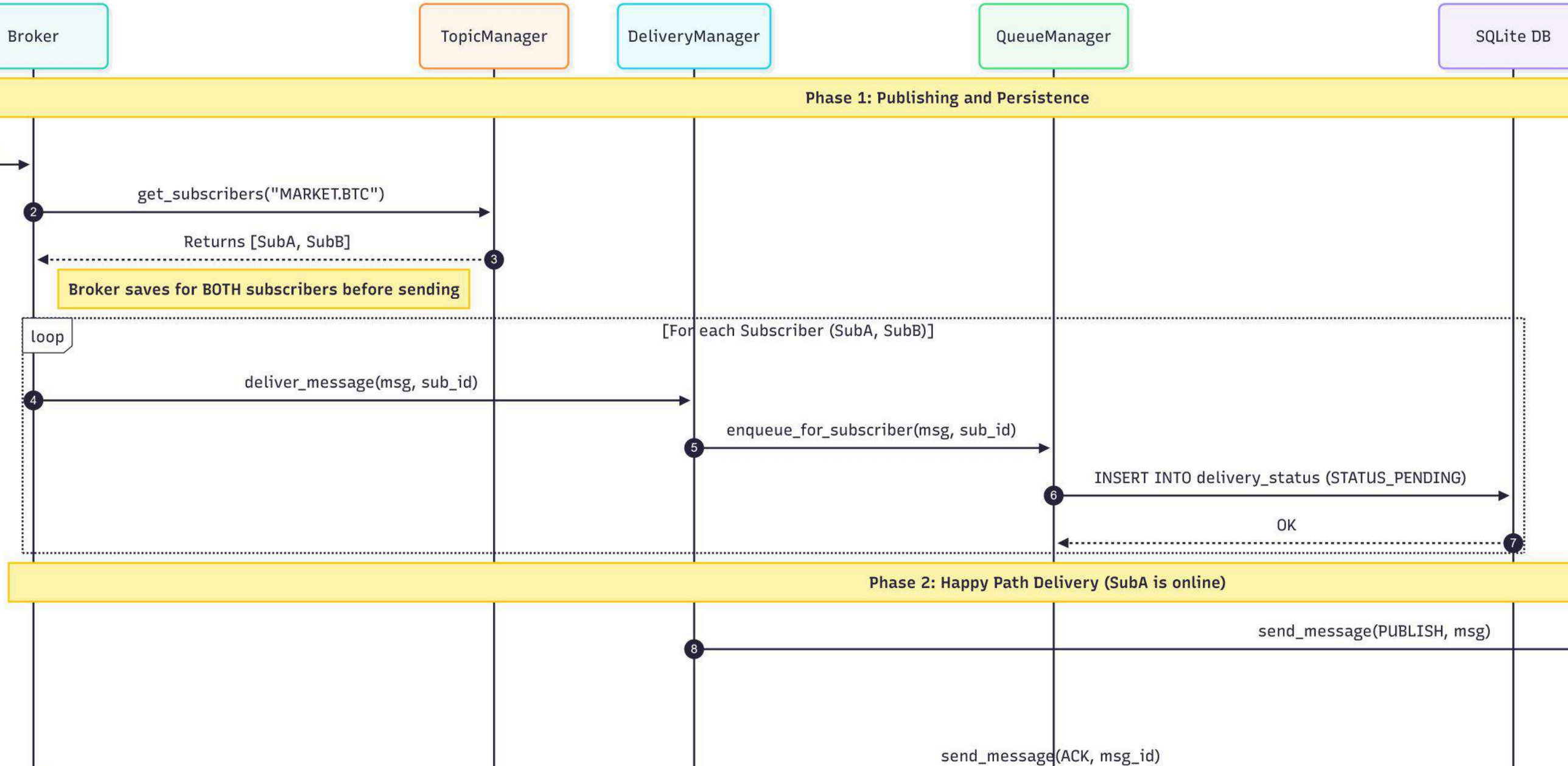
SEQUENCE


Publisher
Broker
TopicManager
DeliveryManager
QueueManager
SQLite DB
SessionManager
SubA (Active)
SubB (Offline)
Phase 1: Publishing and Persistence
send_message(PUBLISH, "MARKET.BTC", payload)
get_subscribers("MARKET.BTC")
Returns [SubA, SubB]
Broker saves for BOTH subscribers before sending
loop
[For each Subscriber (SubA, SubB)]
deliver_message(msg, sub_id)
enqueue_for_subscriber(msg, sub_id)
INSERT INTO delivery_status (STATUS_PENDING)
OK
Phase 2: Happy Path Delivery (SubA is online)
send_message(PUBLISH, msg)
Update Dashboard UI
send_message(ACK, msg_id)
handle_ack(msg_id, "SubA")
mark_delivered()
DELETE FROM delivery_status (For SubA)
OK
Phase 3: Fault Tolerance (SubB reconnects later)
TCP Connect & send_message(CONNECT, "SubB")
register_client("SubB", socket)
get_pending_messages("SubB")
SELECT * FROM delivery_status WHERE sub='SubB'
Returns missed messages
List[Message]
Instantly replays missed messages
loop
[For each missed message]
send_message(PUBLISH, missed_msg)
send_message(ACK, msg_id)
mark_delivered()
DELETE FROM delivery_status (For SubB)

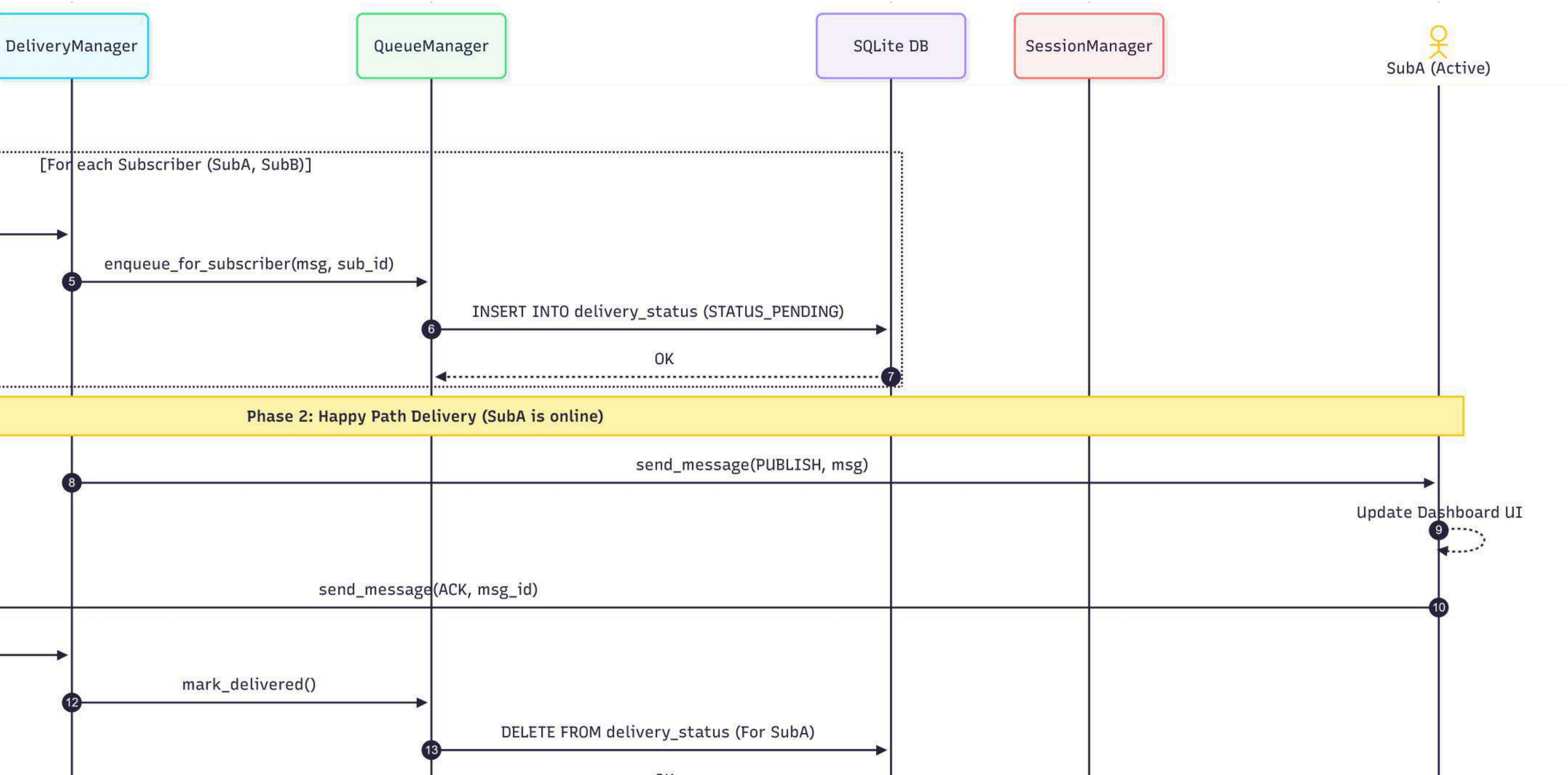
SEQUENCE


Publisher
Broker
TopicManager
DeliveryManager
Que
Phase 1: Publishing and Pers
send_message(PUBLISH, "MARKET.BTC", payload)
1
get_subscribers("MARKET.BTC")
2
Returns [SubA, SubB]
3
Broker saves for BOTH subscribers before sending
loop
[For each Subscriber (SubA, SubB)]
deliver_message(msg, sub_id)
4
enqueue_for_subscriber(msg, sub_id)
5
Phase 2: Happy Path
8
send_mess

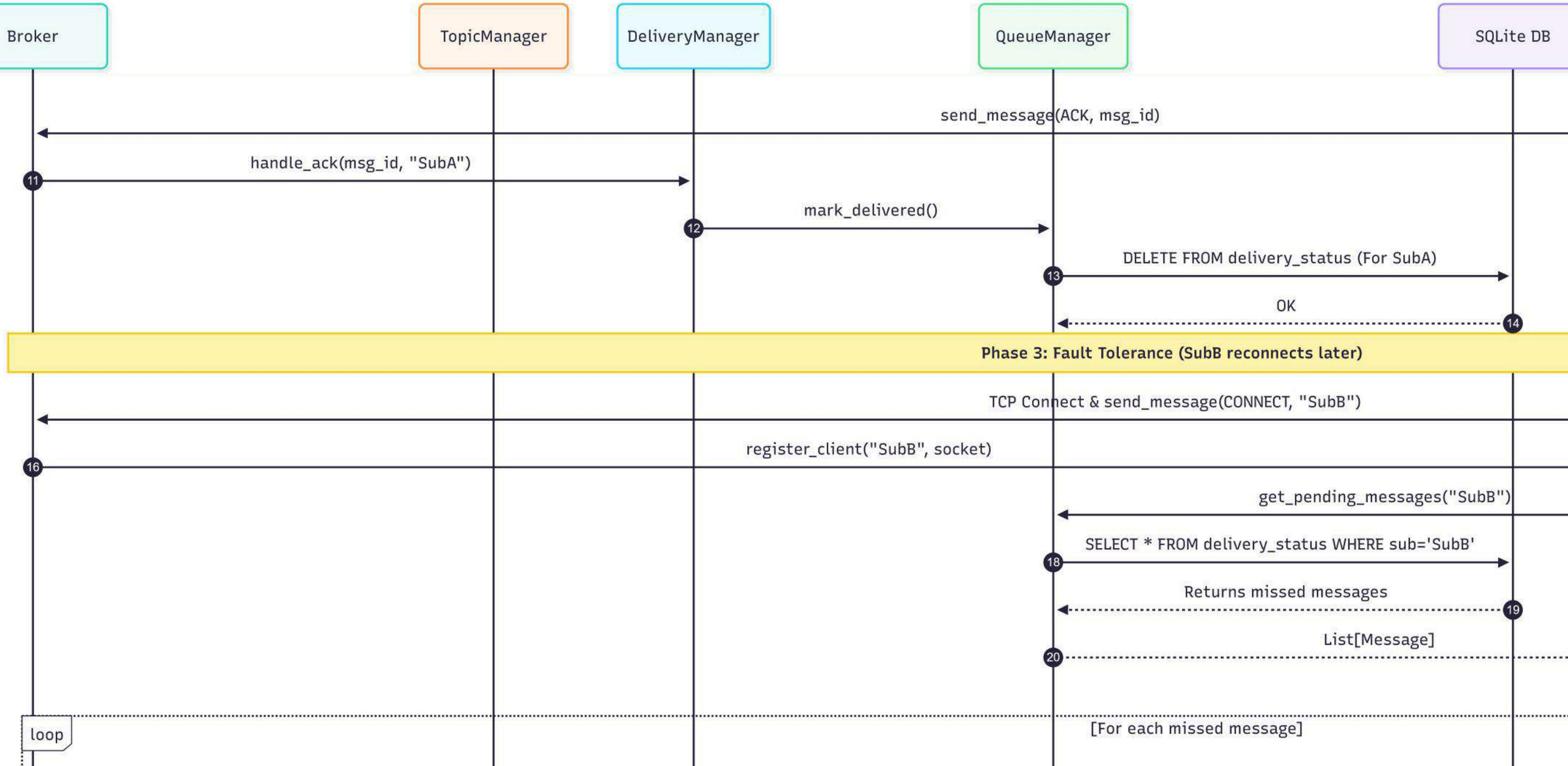
SEQUENCE


Broker
TopicManager
DeliveryManager
QueueManager
SQLite DB
Phase 1: Publishing and Persistence
get_subscribers("MARKET.BTC")
2
Returns [SubA, SubB]
3
Broker saves for BOTH subscribers before sending
loop
[For each Subscriber (SubA, SubB)]
deliver_message(msg, sub_id)
4
enqueue_for_subscriber(msg, sub_id)
5
INSERT INTO delivery_status (STATUS_PENDING)
6
OK
7
Phase 2: Happy Path Delivery (SubA is online)
send_message(PUBLISH, msg)
8
send_message(ACK, msg_id)

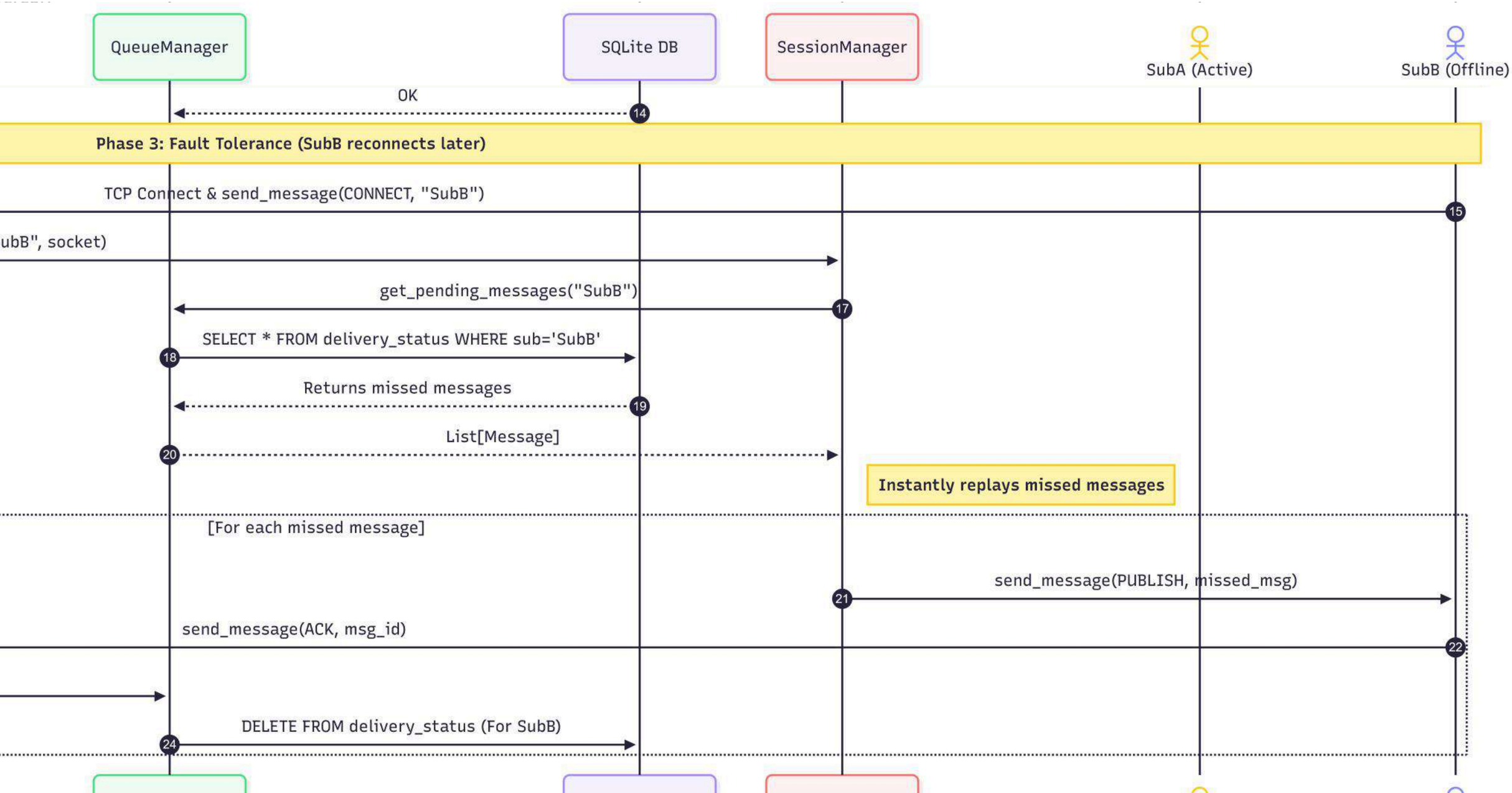
SEQUENCE


DeliveryManager
QueueManager
SQLite DB
SessionManager
SubA (Active)
[For each Subscriber (SubA, SubB)]
enqueue_for_subscriber(msg, sub_id)
5
INSERT INTO delivery_status (STATUS_PENDING)
6
OK
7
Phase 2: Happy Path Delivery (SubA is online)
send_message(PUBLISH, msg)
8
Update Dashboard UI
9
send_message(ACK, msg_id)
10
mark_delivered()
12
DELETE FROM delivery_status (For SubA)
13

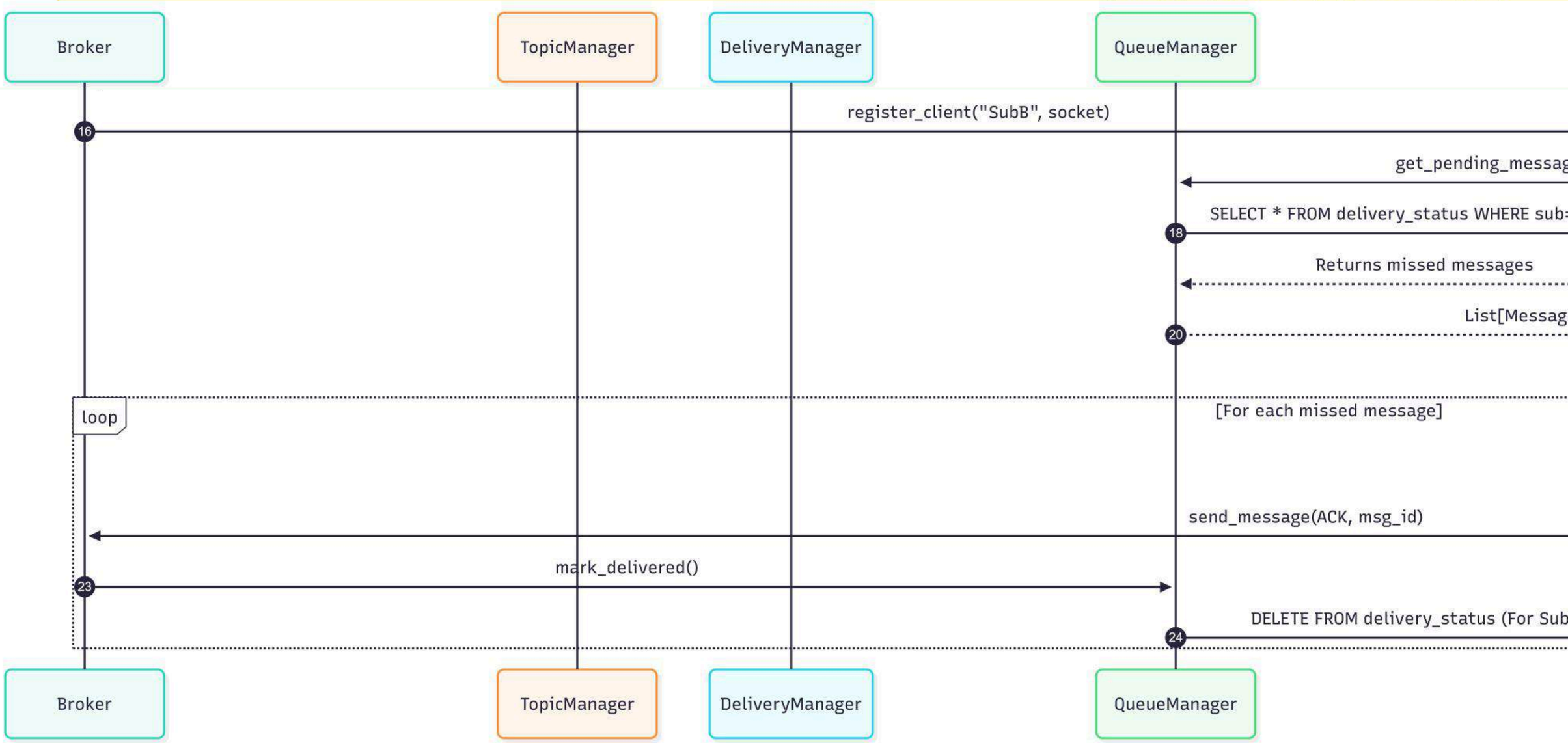
SEQUENCE


Broker
TopicManager
DeliveryManager
QueueManager
SQLite DB
send_message(ACK, msg_id)
handle_ack(msg_id, "SubA")
11
mark_delivered()
12
DELETE FROM delivery_status (For SubA)
13
OK
14
Phase 3: Fault Tolerance (SubB reconnects later)
TCP Connect & send_message(CONNECT, "SubB")
register_client("SubB", socket)
16
get_pending_messages("SubB")
SELECT * FROM delivery_status WHERE sub='SubB'
18
Returns missed messages
19
List[Message]
20
loop
[For each missed message]

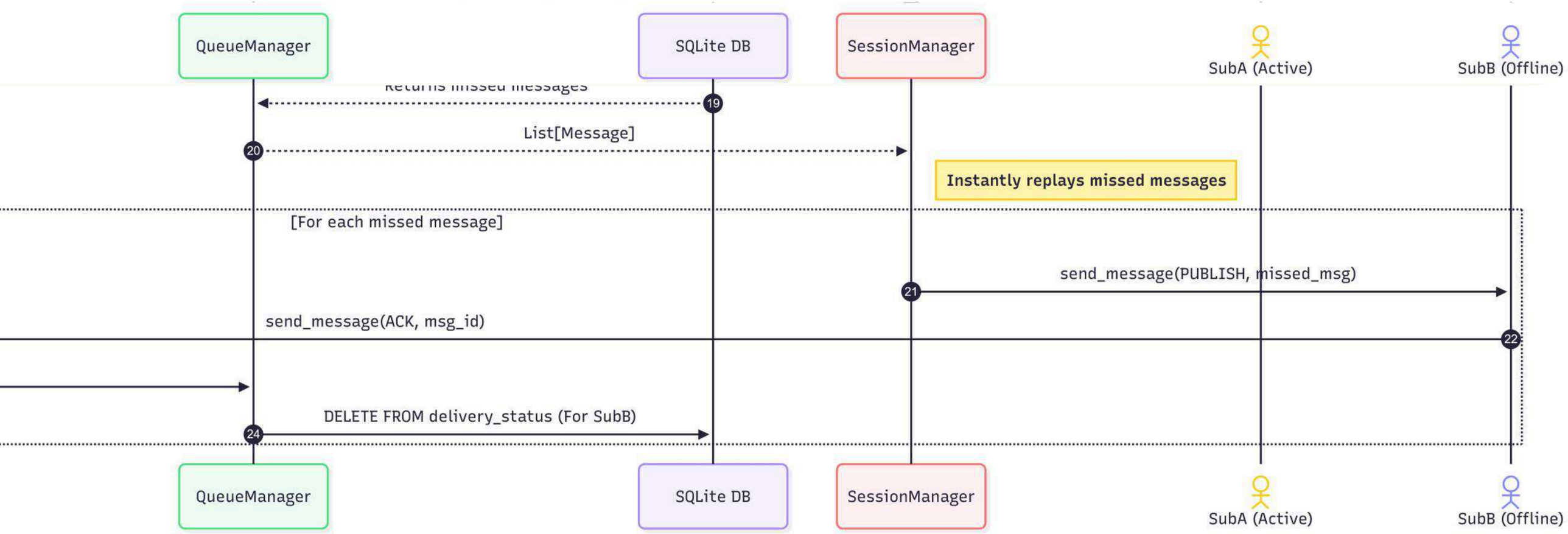
SEQUENCE


QueueManager
SQLite DB
SessionManager
SubA (Active)
SubB (Offline)
OK
14
Phase 3: Fault Tolerance (SubB reconnects later)
TCP Connect & send_message(CONNECT, "SubB")
15
ubB", socket)
get_pending_messages("SubB")
17
SELECT * FROM delivery_status WHERE sub='SubB'
18
Returns missed messages
19
List[Message]
20
Instantly replays missed messages
[For each missed message]
send_message(PUBLISH, missed_msg)
21
send_message(ACK, msg_id)
22
DELETE FROM delivery_status (For SubB)
24

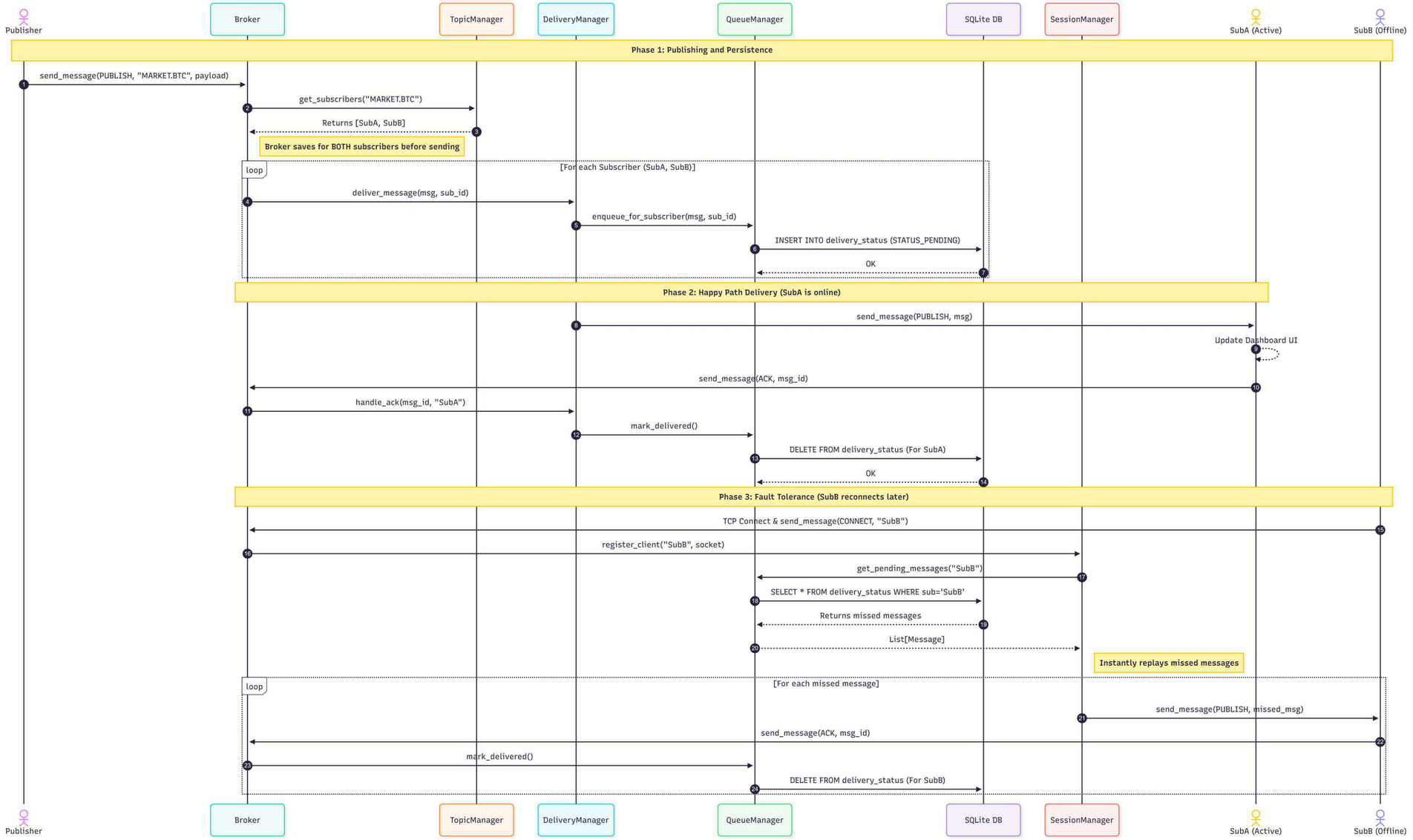
SEQUENCE


Broker
TopicManager
DeliveryManager
QueueManager
register_client("SubB", socket)
16
get_pending_messag
SELECT * FROM delivery_status WHERE sub
18
Returns missed messages
List[Messag
20
loop
[For each missed message]
send_message(ACK, msg_id)
mark_delivered()
23
DELETE FROM delivery_status (For Sub
24
Broker
TopicManager
DeliveryManager
QueueManager

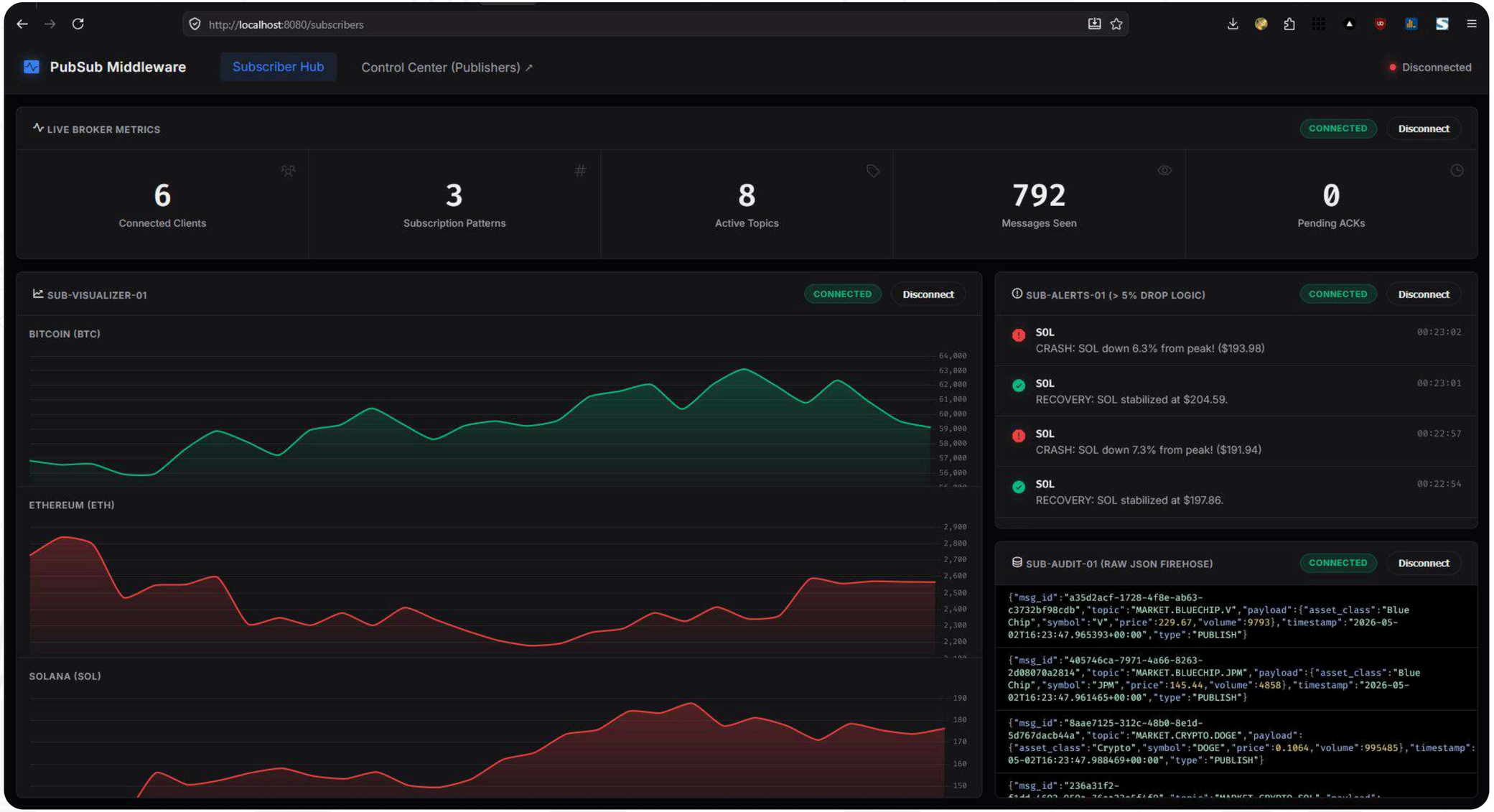
SEQUENCE


QueueManager
SQLite DB
SessionManager
SubA (Active)
SubB (Offline)
Returns missed messages
19
List[Message]
20
Instantly replays missed messages
[For each missed message]
send_message(PUBLISH, missed_msg)
21
send_message(ACK, msg_id)
22
DELETE FROM delivery_status (For SubB)
24
QueueManager
SQLite DB
SessionManager
SubA (Active)
SubB (Offline)

SEQUENCE


Publisher
Broker
TopicManager
DeliveryManager
QueueManager
SQLite DB
SessionManager
SubA (Active)
SubB (Offline)
Phase 1: Publishing and Persistence
send_message(PUBLISH, "MARKET.BTC", payload)
get_subscribers("MARKET.BTC")
Returns [SubA, SubB]
Broker saves for BOTH subscribers before sending
loop
[For each Subscriber (SubA, SubB)]
deliver_message(msg, sub_id)
enqueue_for_subscriber(msg, sub_id)
INSERT INTO delivery_status (STATUS_PENDING)
OK
Phase 2: Happy Path Delivery (SubA is online)
send_message(PUBLISH, msg)
Update Dashboard UI
send_message(ACK, msg_id)
handle_ack(msg_id, "SubA")
mark_delivered()
DELETE FROM delivery_status (For SubA)
OK
Phase 3: Fault Tolerance (SubB reconnects later)
TCP Connect & send_message(CONNECT, "SubB")
register_client("SubB", socket)
get_pending_messages("SubB")
SELECT * FROM delivery_status WHERE sub='SubB'
Returns missed messages
List[Message]
Instantly replays missed messages
loop
[For each missed message]
send_message(PUBLISH, missed_msg)
send_message(ACK, msg_id)
mark_delivered()
DELETE FROM delivery_status (For SubB)

DEMO


http://localhost:8080/subscribers
PubSub Middleware
Subscriber Hub
Control Center (Publishers) ↗
Disconnected
LIVE BROKER METRICS
CONNECTED
Disconnect
6
Connected Clients
3
Subscription Patterns
8
Active Topics
792
Messages Seen
0
Pending ACKs
SUB-VISUALIZER-01
CONNECTED
Disconnect
BITCOIN (BTC)
64,000
63,000
62,000
61,000
60,000
59,000
58,000
57,000
56,000
ETHEREUM (ETH)
2,900
2,800
2,700
2,600
2,500
2,400
2,300
2,200
SOLANA (SOL)
190
180
170
160
150
SUB-ALERTS-01 (> 5% DROP LOGIC)
CONNECTED
Disconnect
SOL
CRASH: SOL down 6.3% from peak! ($193.98)
00:23:02
SOL
RECOVERY: SOL stabilized at $204.59.
00:23:01
SOL
CRASH: SOL down 7.3% from peak! ($191.94)
00:22:57
SOL
RECOVERY: SOL stabilized at $197.86.
00:22:54
SUB-AUDIT-01 (RAW JSON FIREHOSE)
CONNECTED
Disconnect
{"msg_id":"a35d2acf-1728-4f8e-ab63-c3732bf98cdb","topic":"MARKET.BLUECHIP.V","payload":{"asset_class":"Blue Chip","symbol":"V","price":229.67,"volume":9793},"timestamp":"2026-05-02T16:23:47.965393+00:00","type":"PUBLISH"}
{"msg_id":"405746ca-7971-4a66-8263-2d08070a2814","topic":"MARKET.BLUECHIP.JPM","payload":{"asset_class":"Blue Chip","symbol":"JPM","price":145.44,"volume":4858},"timestamp":"2026-05-02T16:23:47.961465+00:00","type":"PUBLISH"}
{"msg_id":"8aae7125-312c-48b0-8e1d-5d767dacb44a","topic":"MARKET.CRYPTO.DOGE","payload":{"asset_class":"Crypto","symbol":"DOGE","price":0.1064,"volume":995485},"timestamp":05-02T16:23:47.988469+00:00","type":"PUBLISH"}
{"msg_id":"236a31f2-